

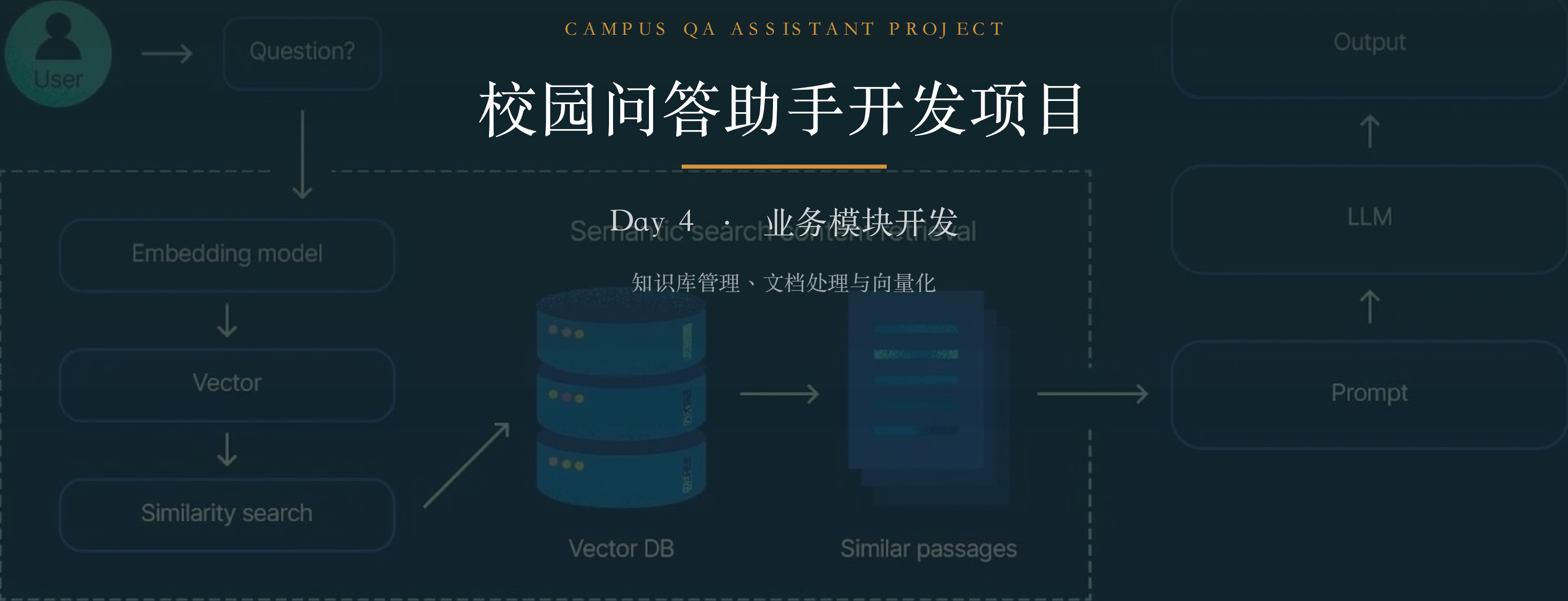
Retrieval augmented generation (RAG)

CAMPUS QA ASSISTANT PROJECT

校园问答助手开发项目

Day 4 · 业务模块开发

知识库管理、文档处理与向量化



今日课程导览

01

知识库 CRUD 模块

数据模型、上传接口、异步处理与状态流转

02

文档处理与向量化

Python 服务：切分 / Embedding / FAISS 与调用

03

前端管理页面

Antd 上传组件、文档列表与处理状态

04

模块架构图

整体数据流、常见问题与今日输出

01

知识库模块与数据模型

文档表设计、整体数据流与向量元数据

知识库模块 CRUD 开发

 文档上传

支持 PDF / Word / TXT
文件大小限制 (50 MB)
存储路径管理
格式校验
POST /api/doc/upload

 文档列表

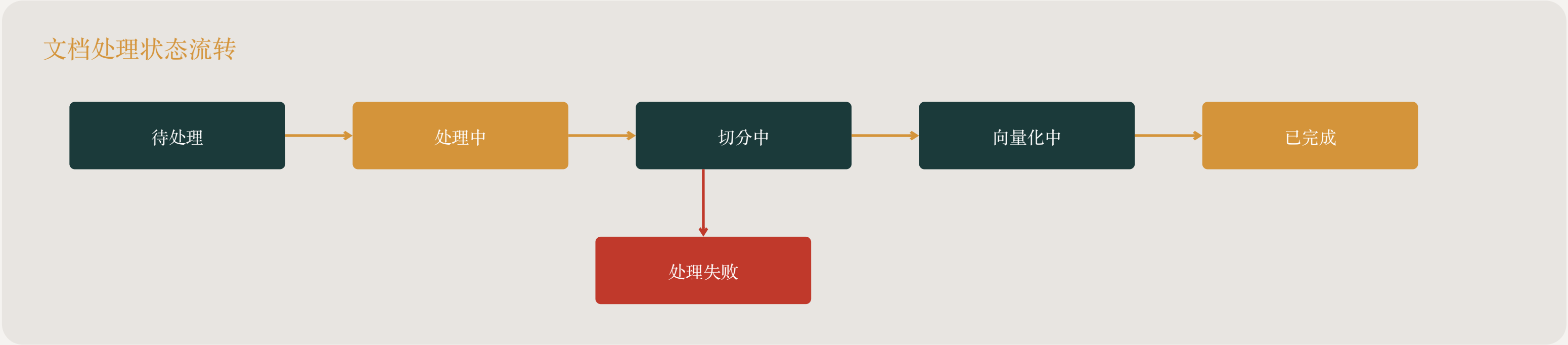
展示所有上传文档
按名称搜索
按状态筛选
分页展示
GET /api/doc/list

 更新 / 重新处理

更新文档内容
触发重新切分
重新向量化
更新向量库
PUT /api/doc/{id}

 文档删除

删除文档记录
删除对应向量数据
清理存储文件
级联处理
DELETE /api/doc/{id}



知识库整体数据流（工程视角）

上传 → 处理 → 向量入库 → 检索，各环节谁负责

- 上传：前端 → `/api/doc/upload` → 存文件 + 写 `kb_document`（状态 = 待处理）
- 处理（异步）：Spring Boot `@Async` → 调用 Python AI 服务 `/process`
- Python：读文件 → 切分 `chunk` → Embedding 向量化 → 写入 FAISS + 元数据
- 完成：回写 `chunkCount`、状态 = 已完成；失败 → 状态 = 处理失败（可重试）
- 检索（Day5）：问题 → Python `/search` → 返回 TopK 文本块 + 来源
- 职责划分：Java 管元数据与流程编排，Python 管向量化与检索

文档表 kb_document : 实体与 Mapper

记录每个文档的元信息与处理状态

```
@Data
@TableName("kb_document")
public class KbDocument {
    @TableId(type = IdType.AUTO)
    private Long id;
    private String title;           // 文档标题
    private String filePath;       // 存储路径
    private String fileType;       // pdf / doc / txt
    private Integer chunkCount;    // 切分块数
    private String status;         // 待处理 / 处理中 / 已完成 / 处理失败
    @TableField(fill = FieldFill.INSERT)
    private LocalDateTime createTime;
}

public interface KbDocumentMapper extends BaseMapper<KbDocument> { }
```


分块与向量的元数据设计

让每个向量都能溯源到原文，是 RAG 可信的关键

- 每个向量保存三要素：`content`（原文）/ `docId`（属于哪个文档）/ `chunkIdx`（第几块）
- 检索返回时带上来源，前端可展示“引用自 XX 文档”
- FAISS 只存向量，原文与元数据另存（`json` / `sqlite` / 表），用下标对齐
- 删除文档时：按 `docId` 删除其全部向量与元数据，保持一致
- 建议再存相似度阈值、更新时间，便于索引重建与效果调优

02

文档上传与处理（后端）

上传接口、异步处理、状态流转与调用 AI 服务

文档上传接口 (MultipartFile)

接收文件 → 校验 → 存储 → 触发处理

```

                                                                    @RestController
@RequestMapping("/api/doc")
public class DocumentController {
    @Resource private DocumentService docService;

    @PostMapping("/upload")
    public Result<Long> upload(@RequestParam("file") MultipartFile file) {
        String type = FileUtil.extName(file.getOriginalFilename());
        if (!List.of("pdf", "doc", "docx", "txt").contains(type))
            throw new BizException(400, "仅支持 pdf/doc/txt");
        if (file.getSize() > 50L * 1024 * 1024)
            throw new BizException(400, "文件不能超过 50MB");
        return Result.success(docService.saveAndProcess(file));
    }
    @DeleteMapping("/{id}")
    public Result<Void> delete(@PathVariable Long id) {
        docService.deleteWithVectors(id);    // 删记录 + 删向量 + 删文件
        return Result.success(null);
    }
}
```

DocumentService : 保存 + 异步处理

上传接口立即返回，处理放线程池异步跑

```

                                                                    @Service
public class DocumentService extends ServiceImpl<KbDocumentMapper, KbDocument> {
    @Resource private AiClient aiClient;           // 调用 Python AI 服务

    public Long saveAndProcess(MultipartFile file) {
        String path = storeToDisk(file);           // 存磁盘 /OSS
        KbDocument doc = new KbDocument();
        doc.setTitle(file.getOriginalFilename());
        doc.setFilePath(path); doc.setStatus("待处理");
        save(doc);
        asyncProcess(doc.getId(), path);           // 异步，接口立即返回
        return doc.getId();
    }
    @Async                                           // 交给线程池，避免阻塞请求
    public void asyncProcess(Long id, String path) {
        try {
            updateStatus(id, "处理中");
            int n = aiClient.process(id, path); // 调 Python : 切分 + 向量化 + 入库
            KbDocument d = getById(id);
            d.setChunkCount(n); d.setStatus("已完成"); updateById(d);
        } catch (Exception e) { updateStatus(id, "处理失败"); }
    }
}
```

处理状态机与异常 / 重试

长耗时任务要异步、可观测、可重试

- 状态流转：待处理 → 处理中 → 已完成 / 处理失败
- 异步处理：上传接口立即返回，处理放线程池，避免请求超时
- 每个环节（读文件 / 切分 / 向量化 / 入库）独立捕获异常，失败即置“处理失败”并记日志
- 支持“重新处理”：重置状态 → 重新切分向量化 → 覆盖旧向量
- 大文件：分批 embedding（batch），避免单次调用过大
- 幂等：重复处理同一文档前，先按 docId 清理其旧向量

Spring Boot 调用 Python AI 服务

用 WebClient 跨服务调用：处理与检索

```

                                                                    @Component
public class AiClient {
    // Python AI 服务 ( 切分 / 向量化 / 检索 )
    private final WebClient web = WebClient.create("http://localhost:8000");

    public int process(Long docId, String filePath) {    // 触发处理
        Map<String, Object> body = Map.of("docId", docId, "path", filePath);
        Map res = web.post().uri("/process").bodyValue(body)
            .retrieve().bodyToMono(Map.class).block();
        return (int) res.get("chunkCount");
    }
    public List<Map> search(String question, int topK) { // 向量检索
        Map<String, Object> body = Map.of("question", question, "topK", topK);
        return web.post().uri("/search").bodyValue(body)
            .retrieve().bodyToFlux(Map.class).collectList().block();
    }
}

```

03

切分 · 向量化 · 向量库 (Python)

把 Day3 的原理，落成一个 FastAPI AI 服务

文档处理：文本切分（Chunk）

为什么需要切分： LLM 有上下文长度限制，长文档无法一次性处理。将文档切分为小块，便于向量化和检索。

固定长度切分

按字符数 /Token 数切分
优点：简单直接
缺点：可能切断语义
适合结构化数据

按段落切分

以自然段落为单位
优点：保留语义完整
缺点：段落长度不均
适合文章类文档

重叠切分（推荐）

相邻块之间有重叠
优点：避免信息丢失
优点：上下文连贯
综合性能最佳

切分参数配置

`chunk_size = 500` （每个块的大小，可根据文档类型调整）
`chunk_overlap = 50` （相邻块重叠大小，保证上下文连贯）
推荐：使用 LangChain 的 `RecursiveCharacterTextSplitter` 实现智能切分

文本切分实现 (LangChain)

递归切分，中文按标点优先，块间重叠保上下文

```
# ai_service/processor.py
from langchain.text_splitter import RecursiveCharacterTextSplitter

def split_document(text: str):
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=500,          # 每块约 500 字
        chunk_overlap=50,        # 相邻块重叠 50 字，保证上下文连贯
        separators=["\n\n", "\n", "。", "！", "？", " ", ""],
    )
    return splitter.split_text(text)  # -> ["...", "...", ...]
```

chunk_size / overlap 是效果调优的关键参数 (回顾 Day3)

向量化处理（Embedding）

Embedding 是将文本转换为高维向量的技术
语义相似的文本在向量空间中距离相近，这是 RAG 检索的基础

Embedding 模型选择

OpenAI text-embedding-ada-002	OpenAI v3 text-embedding-3-small	BAAI BGE 系列（开源）	Moka AI M3E（中文优化）
----------------------------------	-------------------------------------	--------------------	----------------------

向量化流程

文本块 → Embedding API 调用
→ 获取向量（通常 1536 维）

批量处理

使用 batch embedding 提高效率
减少 API 调用次数

向量存储

每条记录保存：

- 文本块原文（content）
- 向量（vector: float[]）
- 元数据（doc_id, chunk_idx）

便于检索后溯源到原始文档

向量化实现 (Embedding)

中文推荐 BGE / M3E ; 归一化后用内积等价余弦

```
# ai_service/embedding.py

from sentence_transformers import SentenceTransformer
import numpy as np

# 中文推荐 BGE / M3E ; 也可换成 OpenAI text-embedding-3
model = SentenceTransformer("BAAI/bge-small-zh-v1.5")

def embed(texts: list[str]) -> np.ndarray:
    vecs = model.encode(texts, normalize_embeddings=True) # 归一化
    return np.array(vecs, dtype="float32")               # (n, 512)
```

向量库构建（FAISS）

FAISS（Facebook AI Similarity Search）是 Facebook 开源的高效相似度搜索库，支持海量向量数据的快速检索

FAISS 索引类型

Flat（暴力搜索）
精确匹配，结果最准确
速度较慢，适合小规模数据

IVF（倒排文件）
分区搜索，平衡精度与速度
适合中等规模数据

HNSW（图索引）
近似搜索，速度最快
适合大规模数据

使用流程

1. 创建索引：指定向量维度
2. 添加向量：批量 add
3. 搜索：传入查询向量 + TopK
4. 持久化：保存为文件

本项目选择

IndexFlatIP
内积相似度计算
适合小规模数据（<10 万条）
精确搜索，实现简单

构建 FAISS 索引 + 持久化

IndexFlatIP 精确检索, add 后务必 write_index 落盘

```
# ai_service/vector_store.py

import faiss, numpy as np, os
INDEX_PATH = "data/faiss.index"

def build_or_add(vectors: np.ndarray, metas: list[dict]):
    dim = vectors.shape[1]
    if os.path.exists(INDEX_PATH):
        index = faiss.read_index(INDEX_PATH)
    else:
        index = faiss.IndexFlatIP(dim) # 内积相似度, 适合小规模精确检索
    index.add(vectors) # 批量加入向量
    faiss.write_index(index, INDEX_PATH) # 持久化到磁盘
    append_meta(metas) # 保存 原文 /docId/chunkIdx

def search(qvec: np.ndarray, top_k=5):
    index = faiss.read_index(INDEX_PATH)
    scores, ids = index.search(qvec, top_k) # 相似度 + 向量下标
    return scores[0], ids[0]
```

FastAPI 暴露 /process 与 /search

AI 服务的两个入口：处理文档、向量检索

```
# ai_service/main.py

from fastapi import FastAPI
from processor import split_document
from embedding import embed
import vector_store as vs
app = FastAPI()

@app.post("/process")          # 供 Spring Boot 调用：切分 + 向量化 + 入库
def process(req: dict):
    text = read_file(req["path"])
    chunks = split_document(text)
    metas = [{"docId": req["docId"], "chunkIdx": i, "content": c}
              for i, c in enumerate(chunks)]
    vs.build_or_add(embed(chunks), metas)
    return {"chunkCount": len(chunks)}

@app.post("/search")          # 供 Day5 的 RAG 检索调用
def search(req: dict):
    scores, ids = vs.search(embed([req["question"]]), req.get("topK", 5))
    return [vs.meta(i) | {"score": float(s)} for s, i in zip(scores, ids)]
```

04

前端：知识库管理

Antd 上传组件、文档列表与实时处理状态

知识库管理后台页面

页面布局：上传区域（拖拽上传） + 文档列表表格
展示文件名、类型、大小、处理状态、上传时间

 上传功能

点击上传和拖拽上传
显示上传进度条
格式校验（pdf/doc/txt）
大小限制提示

 操作功能

查看详情
重新处理
删除文档
批量操作

 状态管理

实时刷新处理状态
失败时显示错误信息
处理中禁用操作
完成自动刷新列表

处理状态展示（Steps/Progress）

使用 Steps 组件展示处理流程：上传 → 提取 → 切分 → 向量化 → 完成

每个步骤显示对应状态：等待 / 进行中 ● / 完成 ✓ / 失败 ✗

文档上传组件 (Antd Upload)

限制格式与大小，带 Token，上传后自动处理

```
// src/pages/KnowledgeBase.tsx
import { Upload, Button, message } from "antd";
import { UploadOutlined } from "@ant-design/icons";

const uploadProps = {
  name: "file", action: "/api/doc/upload",
  accept: ".pdf,.doc,.docx,.txt",
  headers: { Authorization: `Bearer ${localStorage.getItem("token")}` },
  beforeUpload: (file: File) => {
    const ok = file.size / 1024 / 1024 < 50; // 限制 50MB
    if (!ok) message.error("文件不能超过 50MB");
    return ok || Upload.LIST_IGNORE;
  },
  onChange(info: any) {
    if (info.file.status === "done") message.success("上传成功，正在处理");
    if (info.file.status === "error") message.error("上传失败");
  },
};

export const UploadArea = () => (
  <Upload {...uploadProps}>
    <Button icon={<UploadOutlined />}>上传文档 (PDF/Word/TXT)</Button>
  </Upload>);
```

前端页面：文档上传与管理界面

上传组件

- Ant Design Upload
- 配置 accept （允许格式）
- 配置 maxSize （最大大小）
- beforeUpload 校验
- onChange 进度回调
- 自定义上传请求

文件预览

- PDF 预览：react-pdf
- 文本预览：直接展示
- 预览弹窗 Modal
- 分页加载大文件
- 下载原始文件

用户体验优化

- 上传中：禁用操作按钮，显示 loading 状态，防止重复提交
- 处理完成：自动刷新文档列表，Toast 提示成功
- 错误处理：上传失败、处理失败的错误提示与重试机制
- 空状态：无文档时的友好引导提示

文档列表 + 处理状态

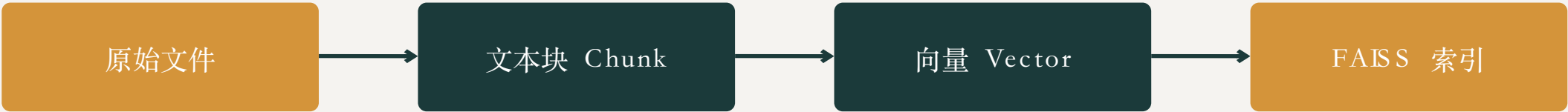
用 Tag 显示状态，处理中定时刷新直至完成

```
const statusColor: any = { "已完成": "green", "处理中": "blue",
  "待处理": "gold", "处理失败": "red" };

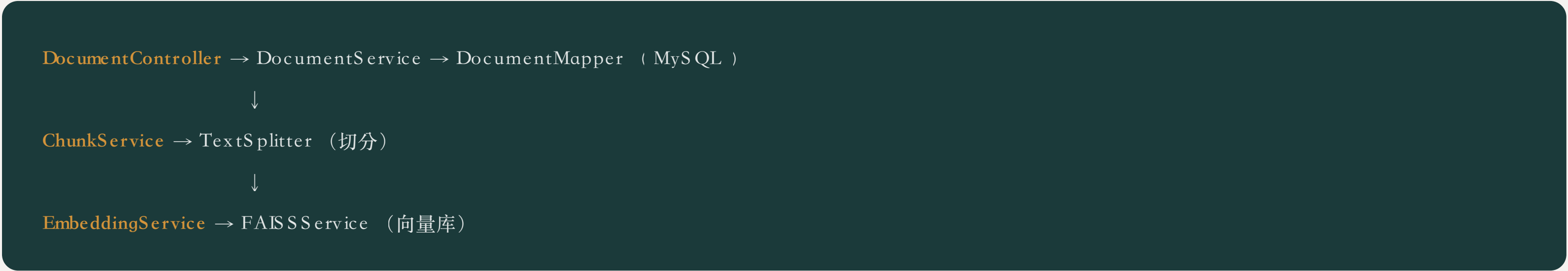
const columns = [
  { title: "文档名", dataIndex: "title" },
  { title: "类型",    dataIndex: "fileType" },
  { title: "块数",    dataIndex: "chunkCount" },
  { title: "状态",    dataIndex: "status",
    render: (s: string) => <Tag color={statusColor[s]}>{s}</Tag> },
  { title: "操作",    render: (_, r: any) =>
    <a onClick={() => del(r.id)}>删除</a> },
];
// 处理中时每 3 秒刷新一次，直到全部完成
useEffect(() => {
  const t = setInterval(() => {
    if (data.some(d => ["处理中", "待处理"].includes(d.status))) load();
  }, 3000);
  return () => clearInterval(t);
}, [data]);
```

知识库模块架构图

数据流转



组件关系



异常处理

每个环节独立捕获异常，任一环节失败时更新文档状态为 "处理失败"，记录错误日志，支持重新处理

常见问题与调试

知识库模块最容易踩的坑

- 上传 413 / 超时：调大网关与后端上传体积上限，前端加进度条
- 中文乱码：读取按 UTF-8 ；PDF 用 pdfplumber / PyMuPDF 提取文本
- 向量维度不匹配：更换 embedding 模型后必须重建索引（维度变了）
- 检索结果不相关：检查是否归一化、chunk 是否过大、TopK 是否合理
- Python 服务连不上：确认 8000 端口、跨服务超时与重试
- FAISS 索引丢失：每次 add 后务必 write_index 持久化到磁盘

Day 4 输出要求

 代码提交
GitLab 提交知识库模块代码

 日报
记录今日完成工作内容

 知识库数据
FAISS 索引文件，可正常检索

 知识库 CRUD 模块
完整文档管理功能

功能验收标准

- ✓ 可上传 PDF/Word/TXT 文档
- ✓ 文档自动完成切分、向量化、入库
- ✓ 可查看文档列表和处理状态
- ✓ 可删除文档及对应向量数据
- ✓ 向量检索可返回正确结果

Retrieval augmented generation (RAG)

DAY 4 COMPLETED

Day 4 课程小结

